Primary functionalities - Visualization heavy design environment - Parametric-first CAD with constraint-based declarative simulations. - GUI for visualization, not for modelling, to encourage parametric CAD - Composable parts, assemblies, physics models, meshes, materials, etc.

Sample usages - Foucault's Pendulum (3D pendulum): - in free-falling elevator - in falling elevator that goes from zero to terminal velocity - on surface of planet (typical Foucault's pendulum demonstration) - on spacecraft in an elliptical orbit with and without non-relativistic approximations - in a room in the interior of a planet - Relativistic examples - Same examples with and without non-relativistic approximations with trivial configuration change (algebra swap) - Lorentz boosts - Gravity slowing clocks (spacecraft in orbit with faster clock than on surface of planet) - High speed slowing clocks

Detailed functionality - Separation between state equations (equations of motion, etc), telemetry, and initial conditions – all three should be composable from *separate* components. For example, should be flexible enough to create an accelerometer component that provides telemetry for angular and linear acceleration and attach the accelerometer to any other component - Alerts on conditions – components should be able to generate alerts when specific conditions are met. Reactions to these alerts should be configurable starting with general behavior of different severities of alerts (pause simulation on fatal alerts but only log errors, warnings and informative alerts, for example) - Composition of components could include ideas from scene graphs (gltf, USD, etc), including materials and textures, lighting, meshes, etc. - Components should be allowed to contain visualization elements, such as meshes and materials, possibly including meshes from step files - Should be able to update running visualizations, situations requiring a reset should be minimized - Maybe composition might work like aspect-oriented programming, attaching visualization and telemetry components to state components, but still allow for visualization and telemetry components to be part of the state component? - Identification of components should follow some form of hierarchical designation structure but with relative and absolute designations. - Must include swizzles for extracting and recomposing values. Would swizzle ideas be useful for components as well? - Every simulation would need to start with a top-level component, plus initial conditions for every state variable. This could be established through run configurations like Java IDEs, but it should be possible to just a simulation from a component in just a few clicks.

GUI - Web-based - 5 main panes: - Project folders and files tree - Editor pane - Visualization pane - Telemetry pane, showing current values of measurables - Logging pane - All panes should be movable and resizable.

Grammar Ideas

First thoughts: component sphere { visual { icosphere s(.subdivisions = 3, .radius = 5) in material copper; } state { mv<euclid3d, double> x{0}; } }

mate(concentric) spring { const { double k = 5; } param { double length =

3; double radius = 1; } visual { helix h(.radius = radius, .length = length) in material brass; } children { component left; component right; } constrain { right.dd(x, t) = -k * length; left.dd(x, t) = k * length; length = right.x - left.x; } }

component wall { visual { wall w(.length = 1) in material steel; } state { fixed mv<euclid3d, double> x{0}; } }

component hookes_law_demo { mate { wall w; sphere s; spring spring{.left = w, .right = s}; } }

component pendulum { param { fixed double length = 3; } visual { rod r(.radius = 1, .length = length) in material brass; } children { component bob; field g = gravity(.dir = {0, 0, -9.8}); } constrain { bob.dd(x, t) = g(bob.m); length = bob.x; } metric { theta = arccos(dot(bob.x, g.dir) / (len(bob.x) * len(g.dir))) } }

// Ideas based on possible scene rendering

a = mesh { file: local / file / path / mesh.ext; };

b = mesh { ./ path / relative / to / this / part / on / same / transport / mesh.ext; };

c = mesh { ./ local / path / relative / to / this / part / mesh.ext; };

d = mesh { {}; // List of vertices {}; // List of indices };

d = mesh { built_in_mesh_creation_function(); };

material1 = material { built_in_material_creator(); };

material2 = material1; material2.diffuse_color = color{1, .5, .3, 1};

// Ideas related to CAD systems

// New types can be created through inheritance. // Here we create a new type of mate called coincident. // The built-in mate type would have all 6 DOF free. coincident : mate { // Translations along x & y directions are fixed. // Translation along z is still free. x = fixed; y = fixed; // Rotations are all fixed. xy = fixed; xz = fixed; yz = fixed;

first.position = <some mv w / pos & orient>; second.position = <some mv w / pos & orient>; first.component = component1; second.component = component2; };

m1 = mate { };